What is a Binary Heap?

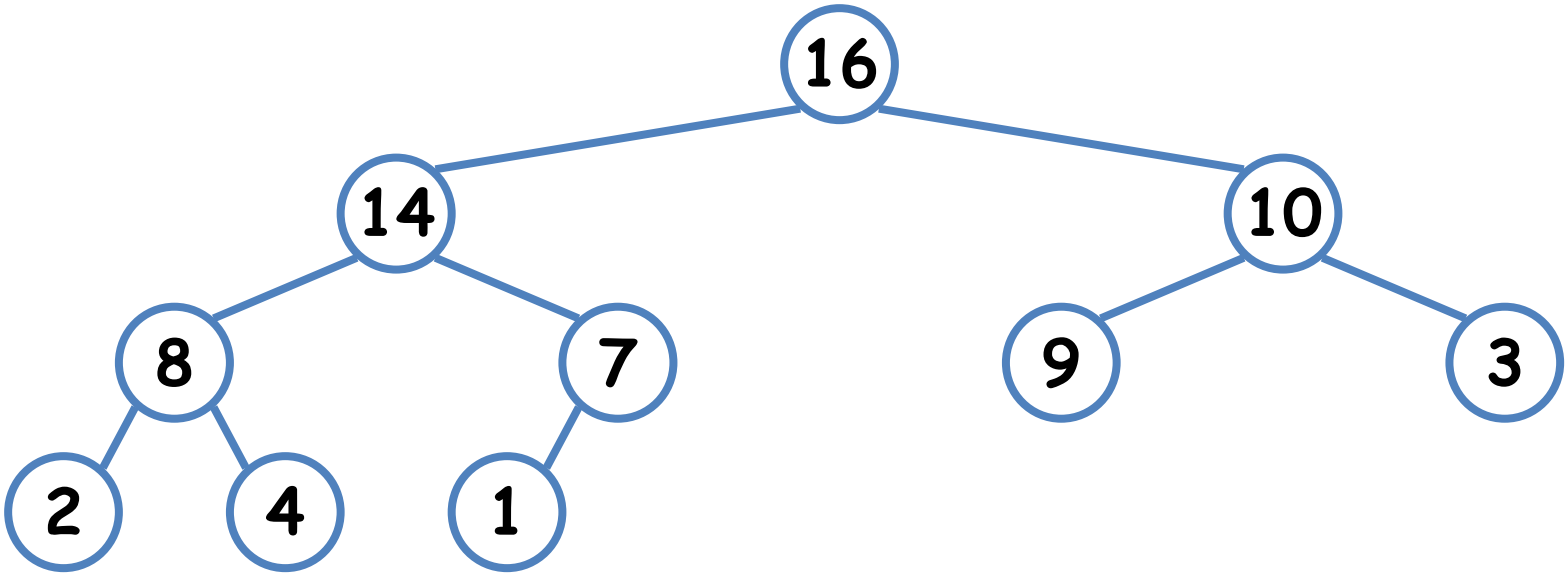
- A **binary heap** is a *complete binary tree* with one (or both) of the following heap order properties:
  - **MinHeap property**: Each node must have a key that is **less or equal** to the key of each of its children.
  - **MaxHeap property**: Each node must have a key that is **greater or equal** to the key of each of its children.

- A binary heap satisfying the MinHeap property is called a **MinHeap**.

- A binary heap satisfying the MaxHeap property is called a **MaxHeap**.

Heap

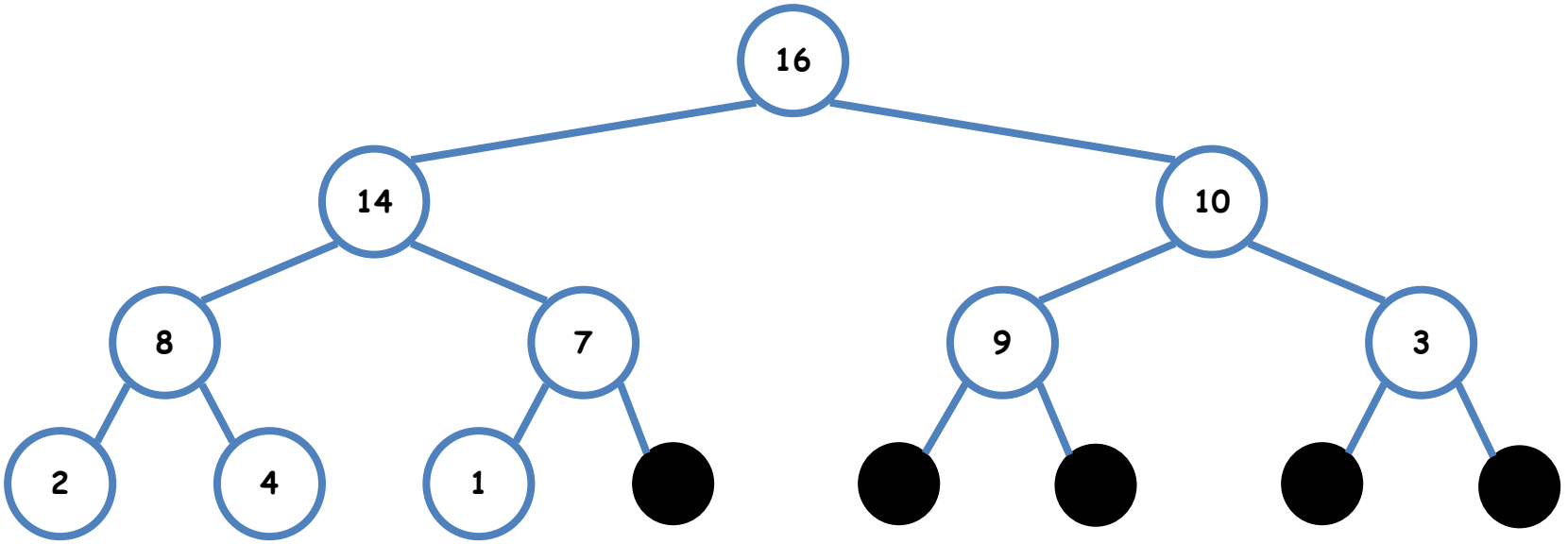
A heap can be seen as a complete binary tree



nearly complete" binary tree ?

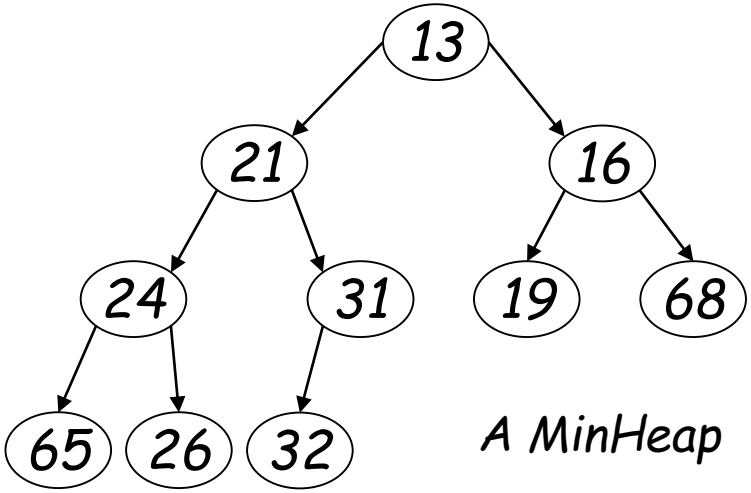
Heap

A *heap* can be seen as a complete binary tree

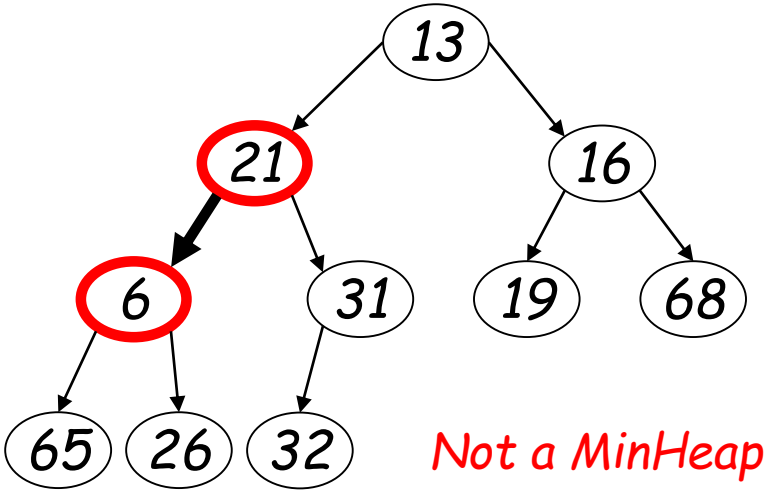


– "nearly complete" binary trees can become complete by making unfilled slots as null pointers

MinHeap and non-MinHeap examples

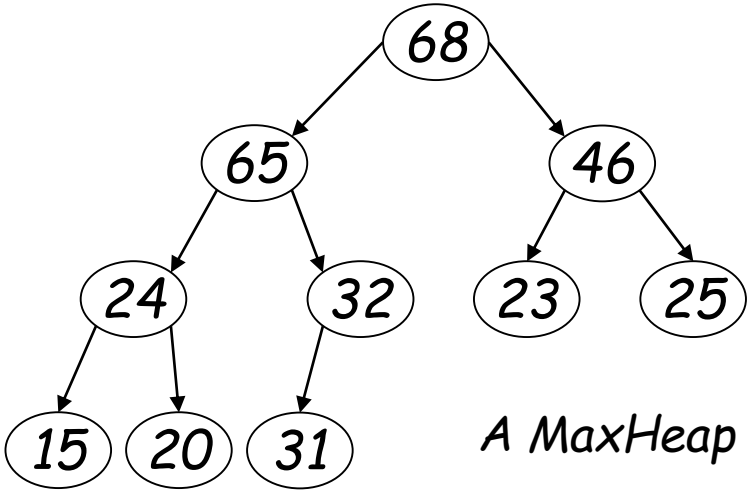


A MinHeap

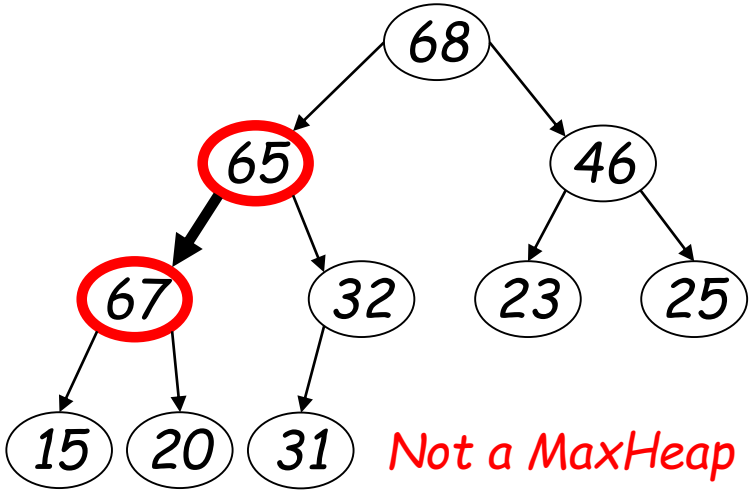


Not a MinHeap

MaxHeap and non-MaxHeap examples



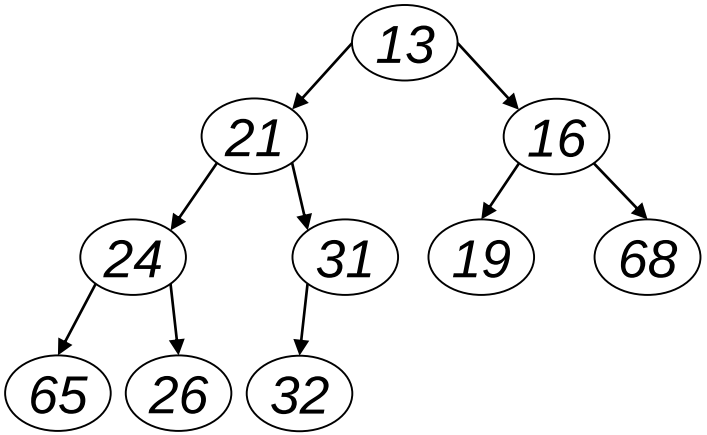
A MaxHeap



Not a MaxHeap

Array Representation of a Binary Heap

A heap is a dynamic data structure that is represented and manipulated more efficiently using an array.



13	21	16	24	31	19	68	65	26	32
1	2	3	4	5	6	7	8	9	10

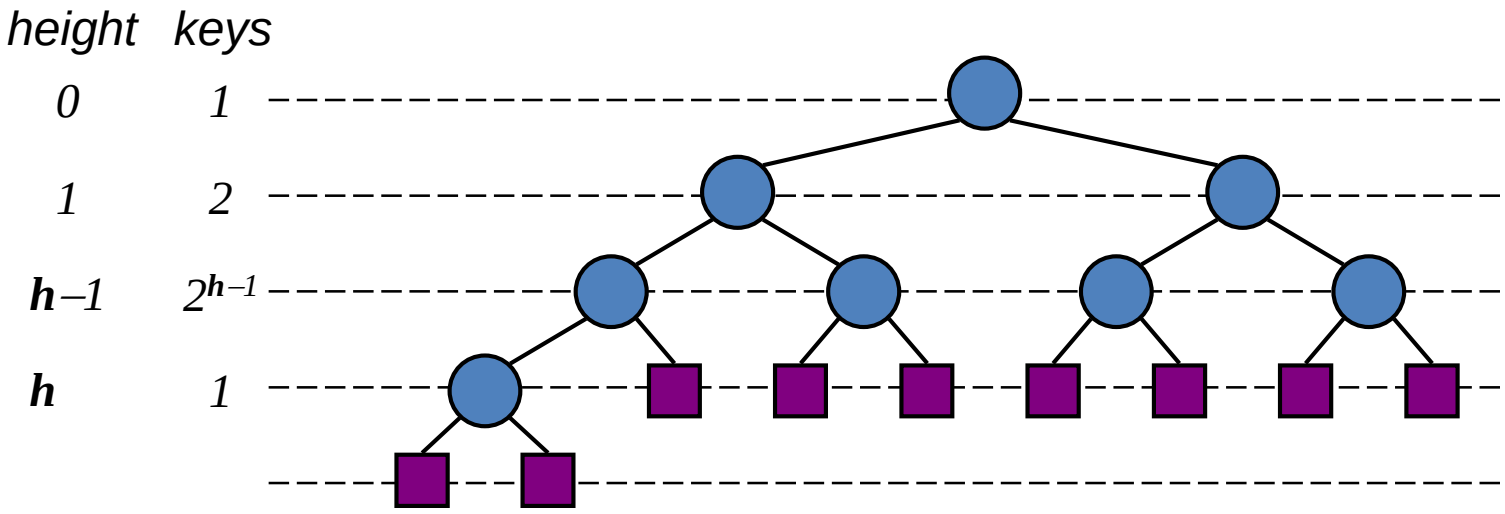
- *The root is array[1]*
- *The parent of array[i] is array[i/2], note: integer divide*
- *The left child, if any, of array[i] is array[2i].*
- *The right child, if any, of array[i] is array[2i+1].*

Height of a Heap

Theorem: A heap storing n keys has height $O(\log n)$

Proof:

- Let h be the height of a heap storing n keys
- Since there are 2^i keys at height $i = 0, \ldots, h-1$ and at least one key at height h, we have $n \geq 1 + 2 + 4 + \ldots + 2^{h-1} + 1$
- Thus, $n \geq 2^h$, i.e., $h \leq \log n$



Heap Operations: Heapify()

Heapify(): **maintain the heap property**

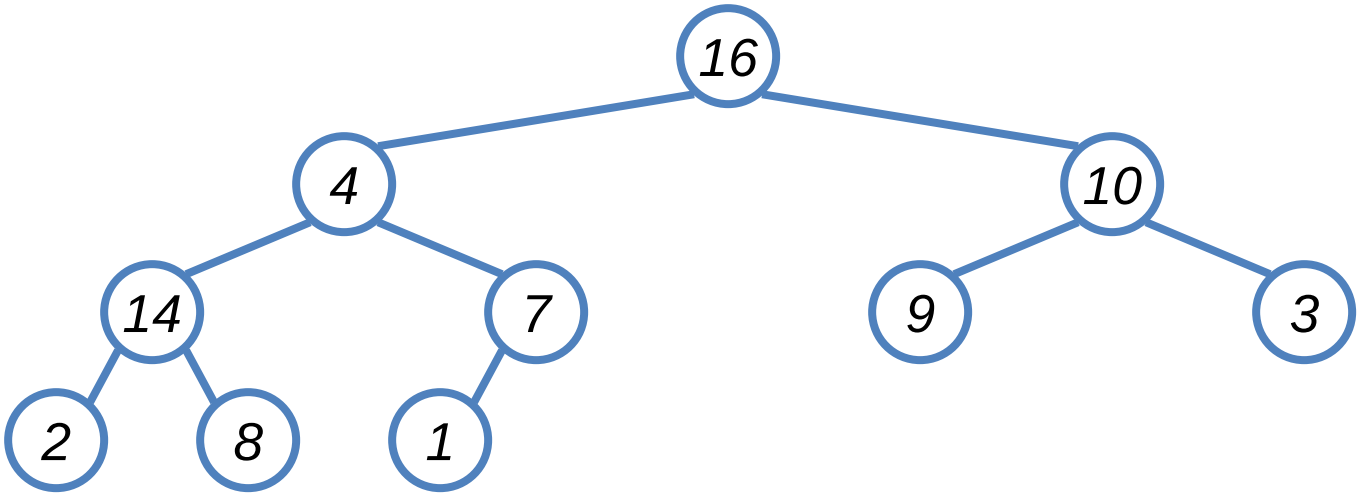
- Given: a node *p* in the heap with children *l* and *r*
- Given: two subtrees rooted at *l* and *r*, assumed to be heaps

- Problem: The subtree rooted at *p* may violate the heap property.
- Action: let the value of the parent node "float down" so the subtree at *p* satisfies the heap property.

Heap Operations: Heapify()

```
Heapify(A, i) //here, i, l, r are indices
{
   l = Left(i); r = Right(i);
   if (l <= heap_size(A) && A[l] > A[i])
       largest = l;
   else
       largest = i;
   if (r <= heap_size(A) && A[r] > A[largest])
       largest = r;
   if (largest != i)
       Swap(A, i, largest);
       Heapify(A, largest);
}
```

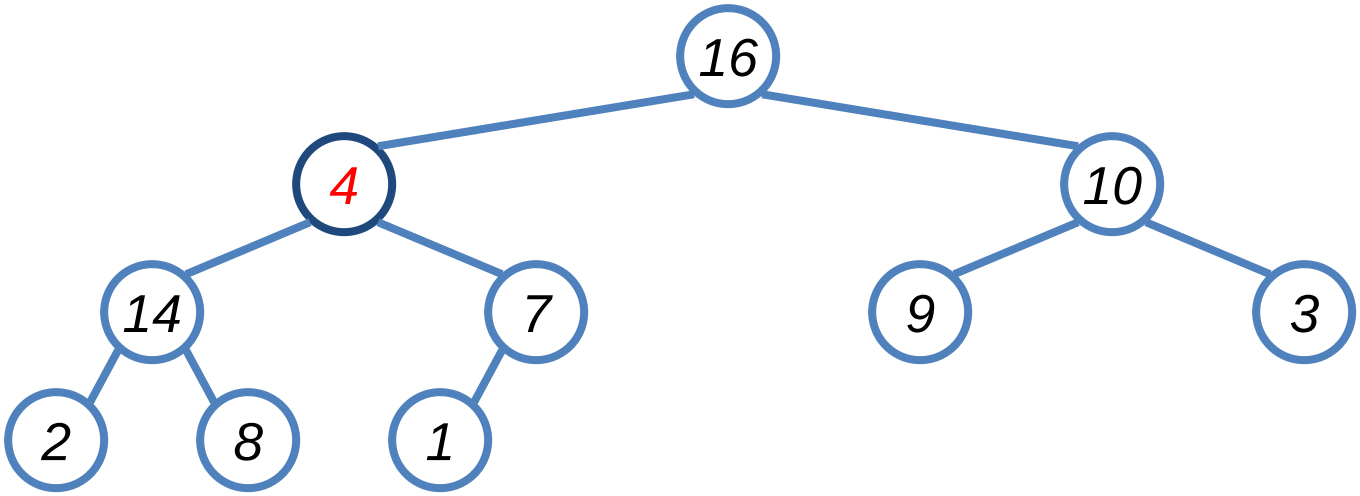
Heapify() Examples



A =

16	4	10	14	7	9	3	2	8	1

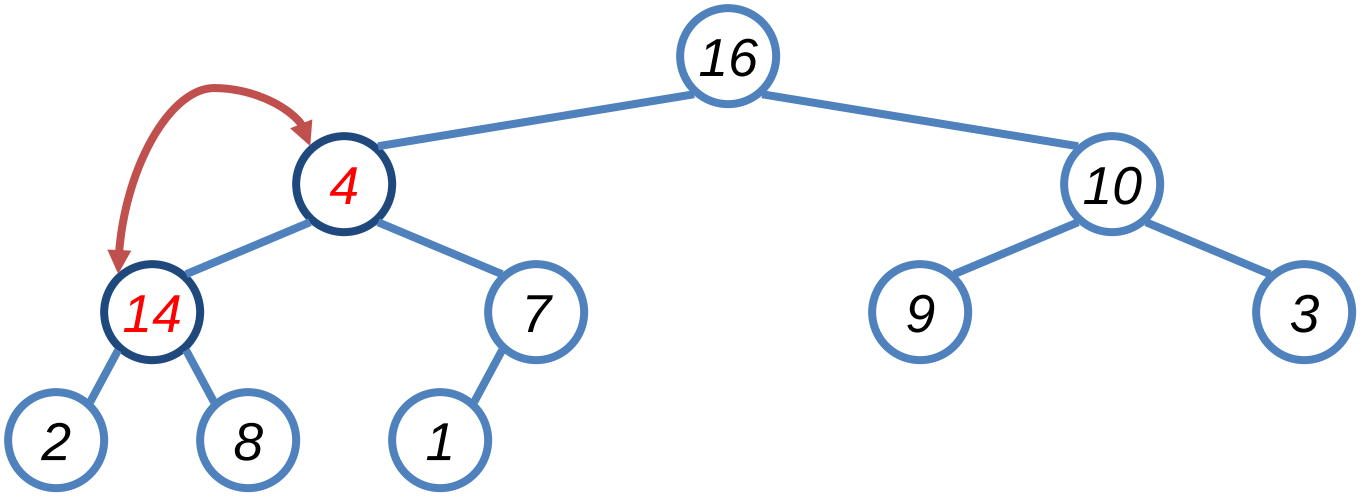
Heapify() Examples

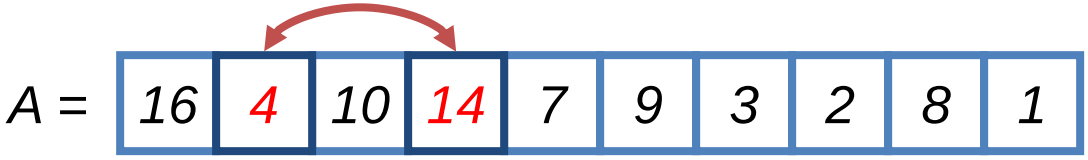


A =

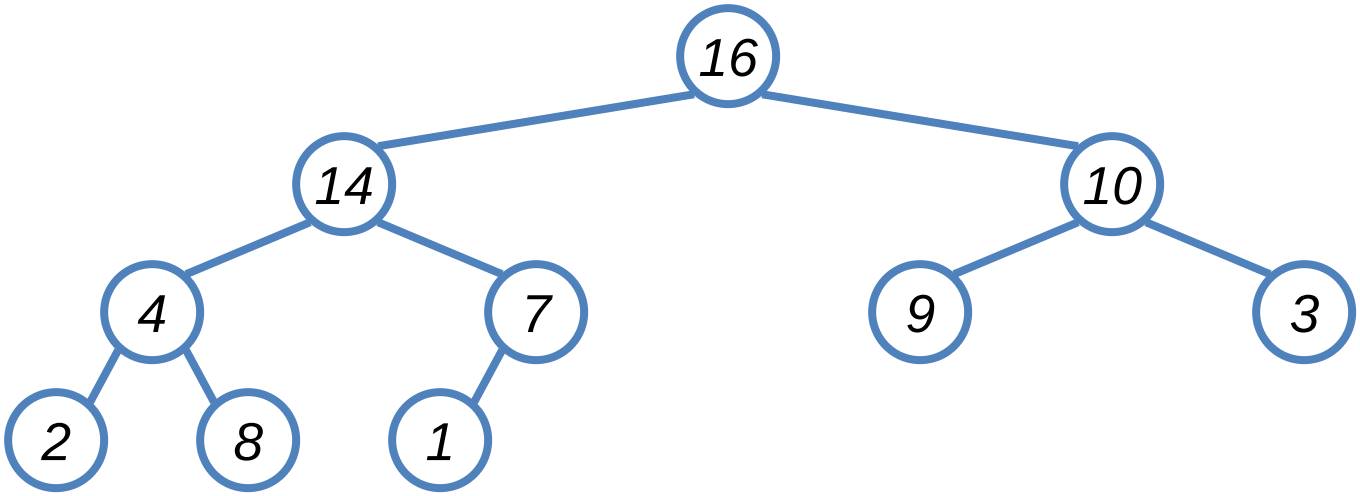
16	4	10	14	7	9	3	2	8	1

Heapify() Examples





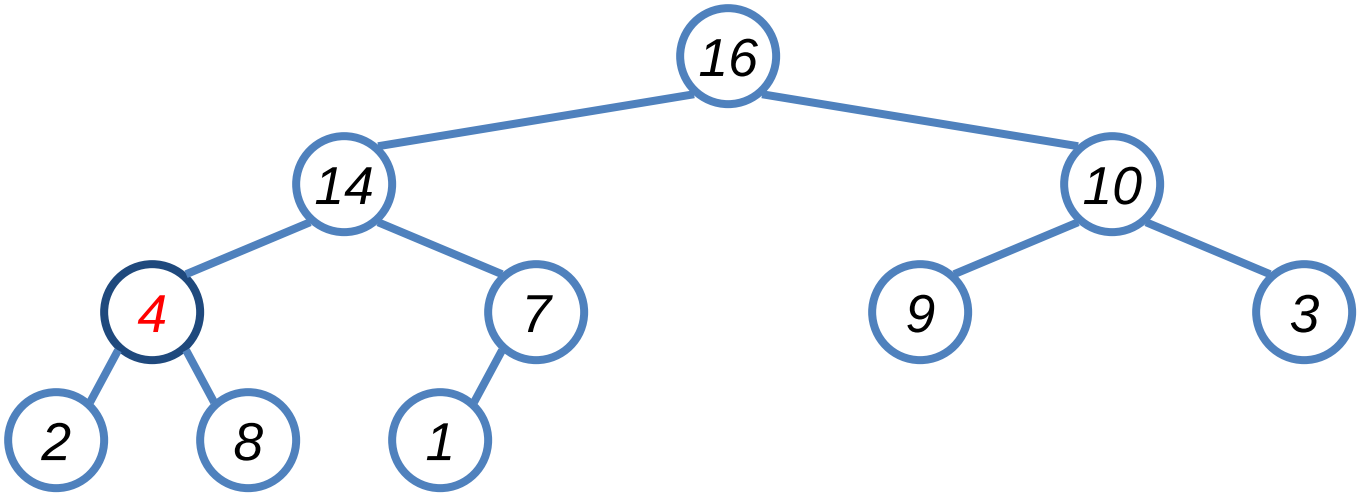
Heapify() Examples



A =

16	14	10	4	7	9	3	2	8	1

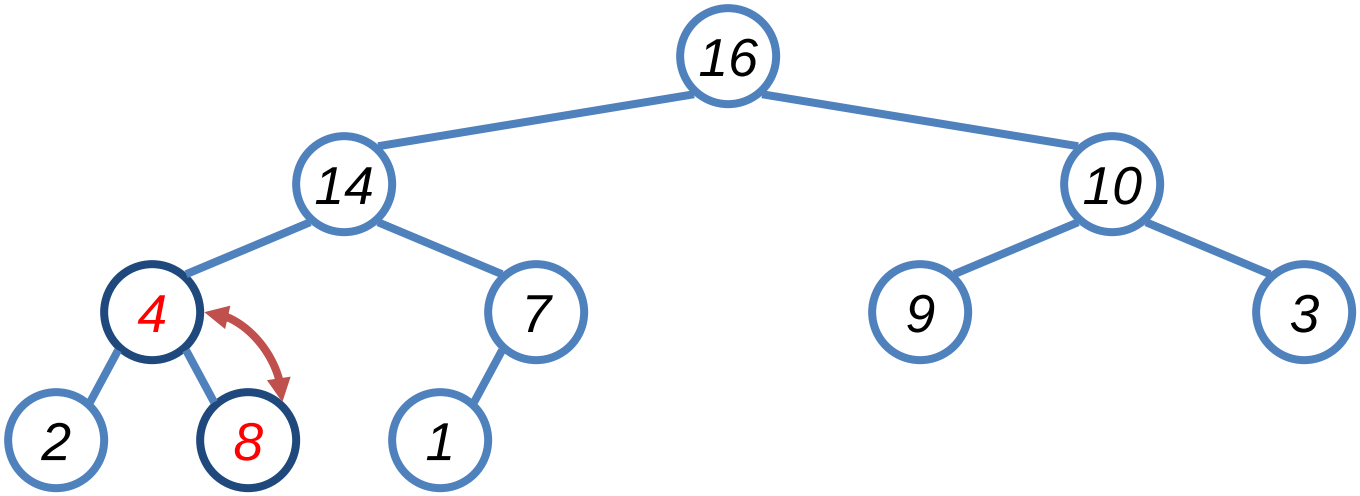
Heapify() Examples

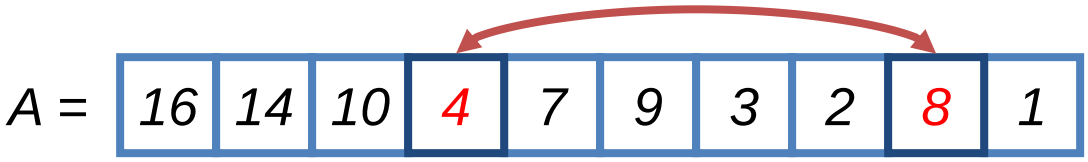


A =

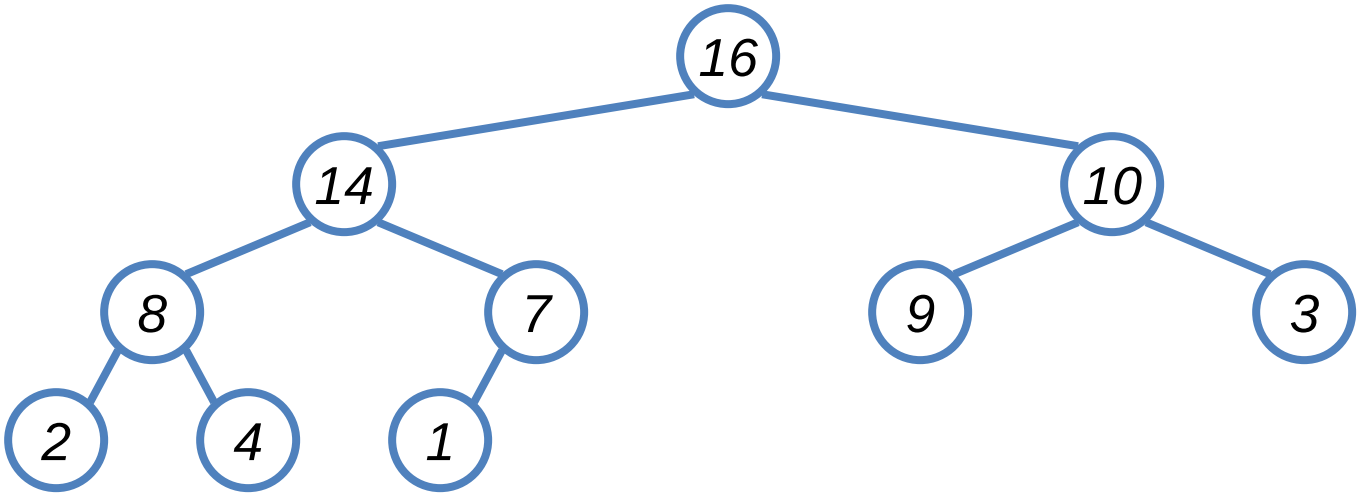
16	14	10	4	7	9	3	2	8	1

Heapify() Examples





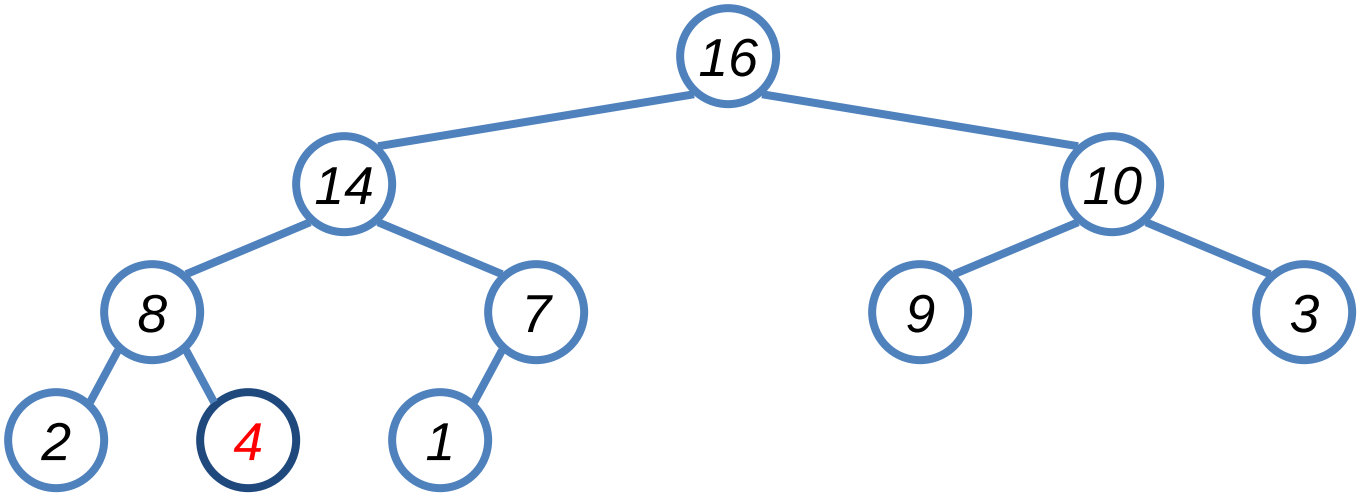
Heapify() Examples



A =

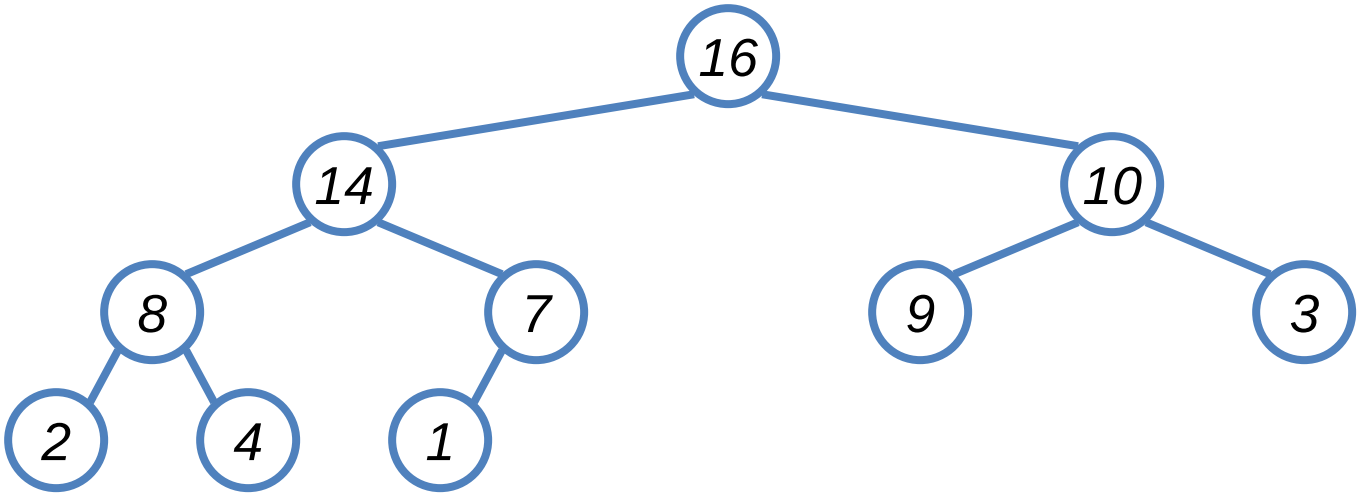
16	14	10	8	7	9	3	2	4	1

Heapify() Examples





Now it is a heap



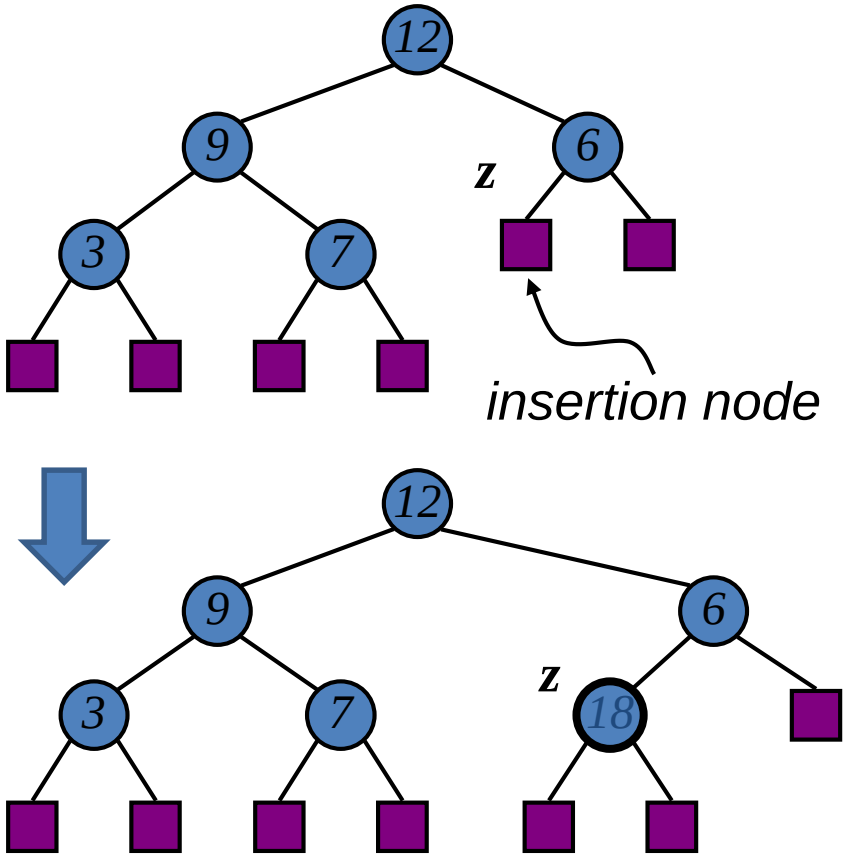
$A =$

16	14	10	8	7	9	3	2	4	1

Insertion of a node into a Heap

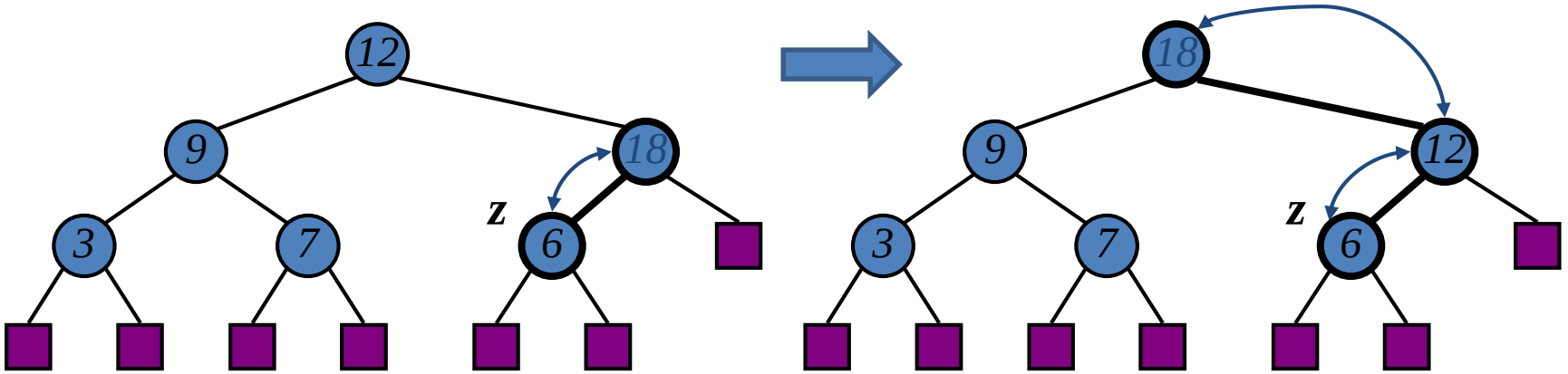
The insertion algorithm consists of three steps

- Find the insertion node ***z*** (the new last node)
- Store ***value k of the node*** at ***z*** and expand ***z*** into an internal node
- Restore the heap-order property



Heapify after insertion

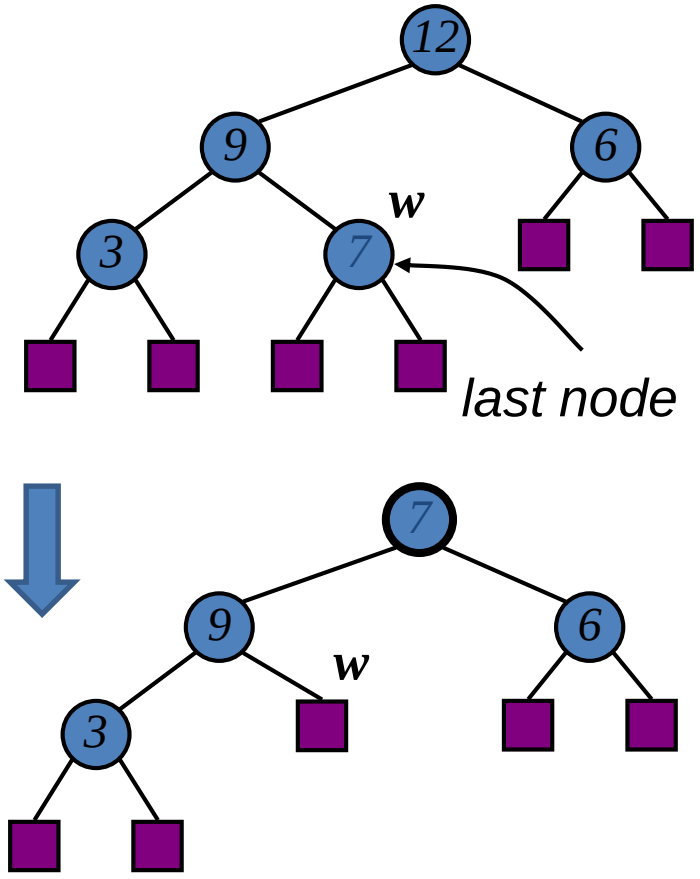
- After the insertion of a new key ***k***, the heap-order property may be violated.
- Heapify() restores the heap-order property by swapping ***k*** along the path upward from the insertion node.
- Heapify() terminates when the key ***k*** reaches the root or a node whose parent has a key larger than or equal to ***k***
- Since a heap has height ***O***(log ***n***), heapify() runs in ***O***(log ***n***) time



Remove key from a Heap

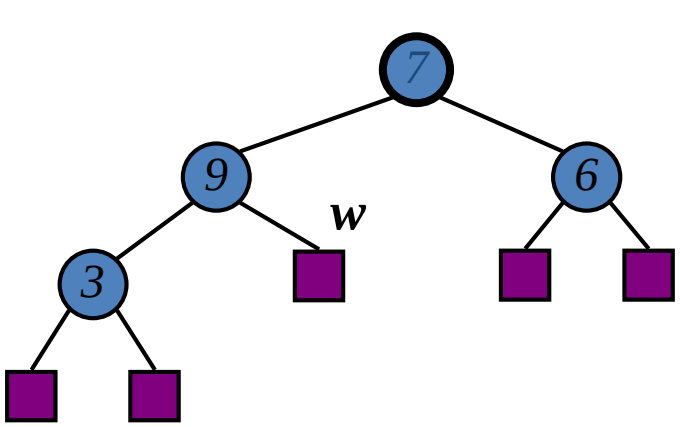
The removal algorithm consists of three steps

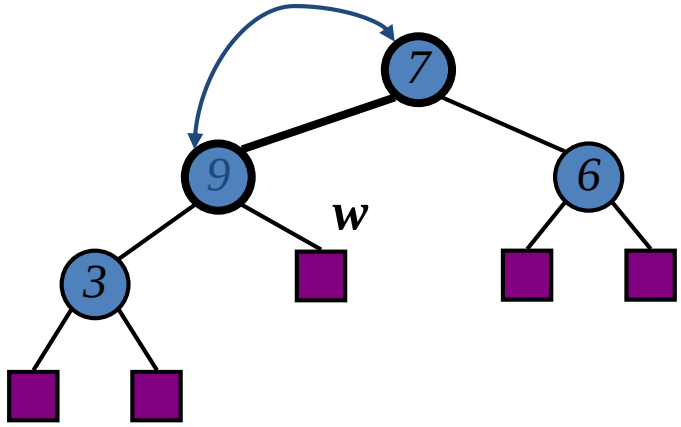
- Extract the key of the root.
- Replace the root key with the key of the last node ***w***
- Compress ***w*** and its children into a leaf.
- Restore the heap-order property.



Heapify() after deletion

- After replacing the root key with the key ***k*** of the last node, the heap-order property may be violated.
- Heapify() restores the heap-order property by swapping the key ***k*** along the path from the root.
- Heapify() terminates when the key ***k*** reaches a leaf or a node whose children have keys smaller than or equal to ***k***
- Since a heap has height ***O***(log ***n***), heapify() runs in ***O***(log ***n***) time





Heap Operations: BuildHeap()

We can build a heap in a bottom-up manner by running **Heapify()** on successive subarrays

- Fact: for an array of length n, half of the elements are heaps and other half are leaves (i.e. elements in range A[1 .. n/2] are heaps, others are leaf nodes).

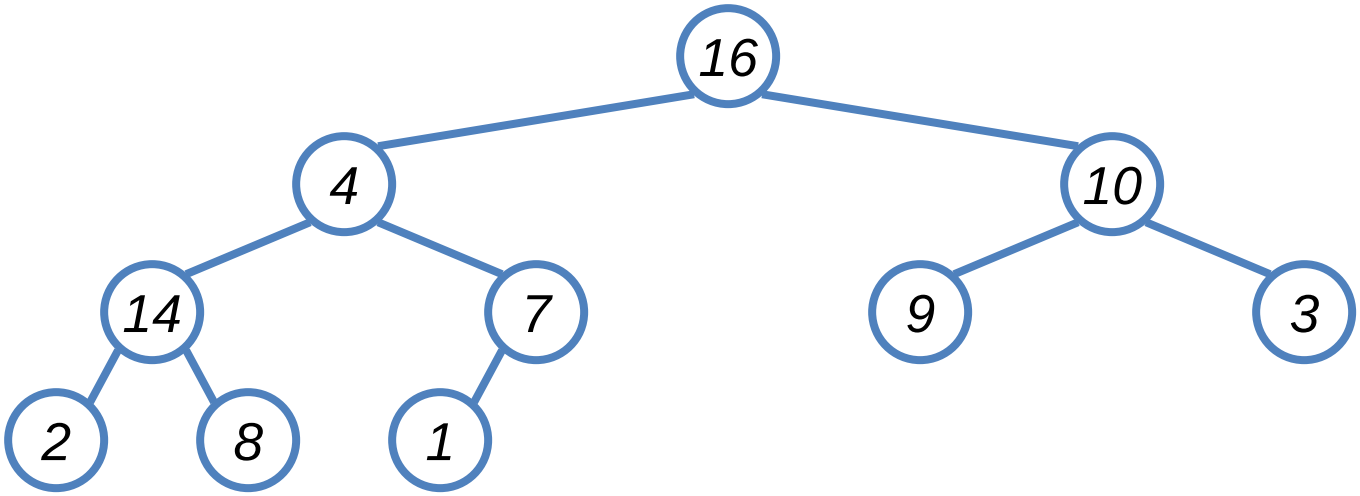
- Therefore:
  - Walk backwards through the array from n/2 to 1, calling **Heapify()** on each node.
  - Order of processing guarantees that the children of node *i* are heaps when *i* is processed.

BuildHeap()

```
// given an unsorted array A, make A a heap

BuildHeap(A)
{
  for (i = ⌊length[A]/2⌋ downto 1)
     Heapify(A, i);
}
```

Build MaxHeap

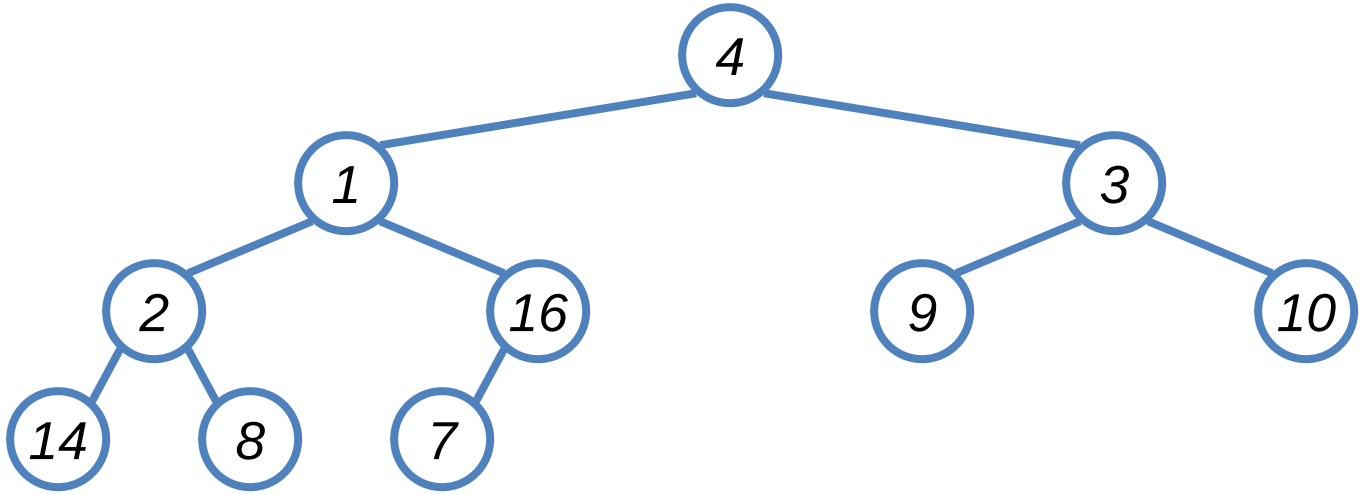


$A =$

16	4	10	14	7	9	3	2	8	1

Build MinHeap

A = {4, 1, 3, 2, 16, 9, 10, 14, 8, 7}



Analyzing BuildHeap()

- Each call to **Heapify()** takes $O(\log n)$ time
- There are $O(n)$ such calls ($\lfloor n/2 \rfloor$ times)
- Thus the running time is $O(n \log n)$

- A tighter bound is $O(n)$